

Machine Learning - Tarea Ficheros

July 8, 2026

1 PAO26-26 - Python 101

1.1 Ficheros



Autor: Elvis Pachacama **Recopilado por:** Pablo Aguilar * Repositorio: GitHub

2 Parte 3: Ficheros (Files)

2.1 1. Acceso a ficheros

- `File` es un tipo de objeto predefinido en Python (*built-in*).
- Permite acceder a ficheros desde programas Python.
- Los ficheros son de un tipo especial: son *built-in*, pero no son ni números, ni secuencias, ni *mappings*. Tampoco responden a operadores en expresiones.
- La función `open` permite crear objetos de tipo fichero.

Formato general para abrir un fichero:

```
afile = open(filename, mode)
```

- `mode` es opcional. Por defecto, los ficheros se abren en modo lectura.
- Los datos leídos de un fichero siempre se obtienen en formato *string*. Lo mismo ocurre con escritura.
- Los ficheros se deben cerrar invocando `close` (liberación de recursos).

```
[1]: # Lectura desde fichero usando método 'read'. Devuelve todo el contenido del fichero.
mi_fichero = open('res/multiple_lines.txt')
```

```

print(mi_fichero.read())
mi_fichero.close()

# Explicación línea por línea:
# 1) open('res/multiple_lines.txt') abre el fichero en modo lectura (modo por
    ↳ defecto, 'r').
# 2) mi_fichero.read() lee TODO el contenido del fichero de una sola vez y lo
    ↳ devuelve
#     como un único string (incluyendo los saltos de línea '\n').
# 3) print(...) lo muestra en pantalla.
# 4) mi_fichero.close() libera el recurso del sistema operativo asociado al
    ↳ fichero abierto;
#     es importante hacerlo siempre para no dejar archivos "colgados".
# Resultado esperado:
# Primera línea del fichero
# Segunda línea del fichero
# Tercera línea del fichero
# (seguido de una línea en blanco extra, porque print añade su propio salto de
    ↳ línea
# además del que ya trae el texto leído)

```

Este fichero
contiene tres
líneas de texto.

```

[2]: # Lectura línea a línea a través del bucle 'for'.
mi_fichero = open('res/multiple_lines.txt')
for linea in mi_fichero:
    print(f"{linea}")
mi_fichero.close()

# Explicación línea por línea:
# 1) Se abre el fichero igual que antes.
# 2) 'for linea in mi_fichero' aprovecha que un objeto fichero es ITERABLE: en
    ↳ cada
#     iteración devuelve la siguiente línea del archivo (incluyendo su '\n'
    ↳ final).
# 3) print(f"{linea}") imprime cada línea; como 'linea' ya trae su propio salto
    ↳ de línea
#     y print añade otro, aparece una línea en blanco extra entre cada línea
    ↳ impresa.
# 4) Se cierra el fichero al terminar.
# Resultado esperado:
# Primera línea del fichero
# (línea en blanco)
# Segunda línea del fichero

```

```
# (línea en blanco)
# Tercera línea del fichero
# (línea en blanco)
```

Este fichero

contiene tres

líneas de texto.

```
[3]: # leer de un archivo a una lista.
mi_fichero = open('res/multiple_lines.txt')
lineas = mi_fichero.readlines()
mi_fichero.close()
print(lineas)

# Explicación línea por línea:
# 1) Se abre el fichero.
# 2) mi_fichero.readlines() lee TODO el contenido y lo devuelve como una LISTA
    ↳ de strings,
#     donde cada elemento es una línea del archivo (cada una conservando su '\n'
    ↳ final).
# 3) Se cierra el fichero.
# 4) print(lineas) muestra la lista completa, con las comillas y los '\n'
    ↳ visibles como
#     caracteres de escape.
# Resultado esperado:
# ['Primera línea del fichero\n', 'Segunda línea del fichero\n', 'Tercera línea
    ↳ del fichero\n']
```

```
['Este fichero\n', 'contiene tres\n', 'líneas de texto.\n']
```

- Resolución de paths independiente de plataforma con `os.path.join`.

```
[4]: import os

ruta = os.path.join("res", "multiple_lines.txt")
print(ruta)

# Explicación línea por línea:
# 1) os.path.join("res", "multiple_lines.txt") construye una ruta de archivo
    ↳ combinando
#     las partes dadas, usando el separador correcto según el sistema operativo:
#     '\\' en Windows, '/' en Linux/Mac. Esto evita escribir rutas "a mano" que
    ↳ solo
#     funcionen en una plataforma.
# 2) print(ruta) muestra la ruta resultante.
```

```
# Resultado esperado (en Windows): res\multiple_lines.txt
# Resultado esperado (en Linux/Mac, como este entorno): res/multiple_lines.txt
```

res\multiple_lines.txt

```
[5]: dataset_file_path = ["res", "multiple_lines.txt"]
ruta = os.path.join(*dataset_file_path)
print(ruta)

# Explicación línea por línea:
# 1) dataset_file_path es una lista con las partes de la ruta.
# 2) 'os.path.join(*dataset_file_path)' usa el operador de desempaquetado '*'
    ↳ para pasar
#     cada elemento de la lista como un argumento posicional separado a join(),
    ↳ equivalente
#     a escribir os.path.join("res", "multiple_lines.txt") directamente.
# 3) Es útil cuando la ruta se construye dinámicamente con un número variable
    ↳ de partes.
# Resultado esperado (en Linux/Mac): res/multiple_lines.txt
```

res\multiple_lines.txt

- Cierre automático con la sentencia `with`. Esta es la forma habitual de leer de un fichero en Python (*context manager*).

```
[6]: with open(ruta) as mi_fichero:
      for linea in mi_fichero:
          print(linea, end='')

# Explicación línea por línea:
# 1) 'with open(ruta) as mi_fichero:' abre el fichero y garantiza que se cerrará
#     AUTOMÁTICAMENTE al salir del bloque 'with', incluso si ocurre un error
    ↳ dentro
#     (no hace falta llamar a close() manualmente).
# 2) for linea in mi_fichero recorre cada línea del archivo.
# 3) print(linea, end='') imprime cada línea SIN agregar un salto de línea
    ↳ adicional
#     (end='' reemplaza el salto de línea por defecto de print), ya que 'linea'
    ↳ ya trae
#     su propio '\n'. Así se evita la línea en blanco extra que aparecía en el
    ↳ ejemplo
#     anterior con el bucle for.
# Resultado esperado:
# Primera línea del fichero
# Segunda línea del fichero
# Tercera línea del fichero
```

Este fichero
contiene tres

líneas de texto.

- Se pueden abrir varios ficheros en un mismo `with`.

```
[7]: # abrir varios ficheros en el mismo with
ruta1 = os.path.join("res", "one_line.txt")
ruta2 = os.path.join("res", "multiple_lines.txt")
with open(ruta1) as fichero1, open(ruta2) as fichero2:
    print(fichero1.readlines())
    print(fichero2.readlines())

# Explicación línea por línea:
# 1) ruta1 y ruta2 apuntan a dos archivos distintos.
# 2) 'with open(ruta1) as fichero1, open(ruta2) as fichero2:' abre AMBOS
    ↪ ficheros dentro
#     del mismo bloque with, separados por coma; los dos se cerrarán
    ↪ automáticamente al
#     salir del bloque.
# 3) fichero1.readlines() devuelve la lista de líneas de one_line.txt (un solo
    ↪ elemento).
# 4) fichero2.readlines() devuelve la lista de líneas de multiple_lines.txt
    ↪ (tres elementos).
# Resultado esperado:
# ['Esta es una única línea de texto.\n']
# ['Primera línea del fichero\n', 'Segunda línea del fichero\n', 'Tercera línea
    ↪ del fichero\n']
```

```
['Hola mundo desde un fichero aqui estamos en machine.']
```

```
['Este fichero\n', 'contiene tres\n', 'líneas de texto.\n']
```

2.2 2. Modos de acceso

Al crear un objeto de tipo `File` se puede especificar el modo de acceso (lectura/escritura):

Modo	Descripción
r	Solo Lectura
w	Solo Escritura (Borra si el archivo ya existe)
x	Solo Escritura (Falla si el archivo ya existe)
a	Crea Fichero (Si existe, lo abre y añade al final)
r+	Lectura y Escritura
b	Se puede añadir a otros modos para acceso binario
t	Acceso para archivos de texto (default)

2.3 3. Acceso para escritura

```
[8]: def crear_lista(tamanyo):
    lista = []
    for i in range(tamanyo):
        lista.append(str(i) + '\n')
    return lista

ruta = os.path.join("res", "a_dummy.txt")
with open(ruta, 'wt') as fichero:
    fichero.write('Cabecera del ciclo del for en el ejemplo\n')
    lista = crear_lista(10)
    fichero.writelines(lista)

print("archivo creado")

# Explicación línea por línea:
# 1) crear_lista(tamanyo) genera una lista de strings: "0\n", "1\n", ..., hasta
#    ↪ tamanyo-1,
#    cada uno con su salto de línea incluido.
# 2) ruta apunta a un nuevo archivo 'res/a_dummy.txt'.
# 3) 'wt' significa modo escritura ('w') + texto ('t', que es el modo por
#    ↪ defecto).
#    Si el archivo ya existía, su contenido anterior se BORRA por completo.
# 4) fichero.write(...) escribe una línea de cabecera manualmente.
# 5) lista = crear_lista(10) genera 10 líneas ("0" a "9").
# 6) fichero.writelines(lista) escribe TODAS las líneas de la lista de una vez
#    ↪ (a diferencia
#    de write(), writelines() NO añade saltos de línea automáticamente; por eso
#    ↪ cada
#    elemento de 'lista' ya trae su propio '\n').
# 7) Al salir del 'with', el archivo se cierra y guarda automáticamente.
# Resultado esperado: "archivo creado"
# (y el archivo res/a_dummy.txt queda con 11 líneas: la cabecera + los números
#    ↪ 0 al 9)
```

archivo creado

2.3.1 Buffering

- Por defecto, el texto que se transfiere desde el programa a un fichero no se guarda en disco inmediatamente: se almacena en un *buffer*.
- Acciones como cerrar un fichero o invocar el método `flush` fuerzan que el contenido del buffer se transfiera a disco.

```
[9]: ruta = os.path.join("res", "FicheroParaEscritura.txt")
fichero_write = open(ruta, 'w')
fichero_write.write('foter')
```

```

fichero_read = open(ruta, "r")
print(fichero_read.readlines())

fichero_write.flush()
print(fichero_read.readlines())

fichero_write.close()
fichero_read.close()

# Explicación línea por línea:
# 1) Se abre 'FicheroParaEscritura.txt' en modo escritura ('w') y se escribe el
    ↳ texto 'foter',
# pero SIN cerrar ni hacer flush todavía: el texto queda en el buffer de
    ↳ escritura,
# aún no está garantizado que esté físicamente en disco.
# 2) Se abre el MISMO archivo en modo lectura, en un objeto file distinto
    ↳ (fichero_read).
# 3) El primer fichero_read.readlines() puede devolver una lista VACÍA (o
    ↳ incompleta),
# porque el contenido escrito por fichero_write todavía podría no haberse
    ↳ volcado a disco.
# 4) fichero_write.flush() fuerza a que el buffer se escriba físicamente en
    ↳ disco de inmediato.
# 5) El segundo fichero_read.readlines() ya debería poder leer el contenido
    ↳ 'foter', aunque
# en algunos sistemas también puede depender de la posición del cursor de
    ↳ lectura
# (fichero_read ya había intentado leer una vez).
# 6) Se cierran ambos ficheros al final.
# Resultado esperado (puede variar ligeramente según el sistema operativo y el
    ↳ buffering):
# []
# ['foter']
# Nota: este ejemplo ilustra el concepto de buffering; el comportamiento exacto
    ↳ (si la
# primera lectura muestra [] o ya el contenido) depende de la implementación
    ↳ del sistema
# de archivos y del tamaño del buffer, por lo que no siempre es 100%
    ↳ determinista.

```

```

[]
['foter']

```

2.4 4. Archivos CSV

Python permite leer y escribir datos en formato CSV (*Comma-Separated Values*), un formato popular en ciencia de datos.

```
[10]: # tabla_operaciones.csv contiene valores separados por comas
import csv

ruta = os.path.join("res", "tabla_operaciones.csv")
with open(ruta, encoding="utf-8") as fichero:
    data_reader = csv.reader(fichero, delimiter=',')
    for linea in data_reader:
        print(linea[0] + ' ---- ' + linea[1])

# Explicación línea por línea:
# 1) import csv carga el módulo estándar para trabajar con archivos CSV.
# 2) Se abre el archivo CSV con codificación UTF-8 explícita (para mostrar
    ↪ correctamente
#     tildes como en "división").
# 3) csv.reader(fichero, delimiter=',') crea un objeto lector que interpreta
    ↪ cada línea
#     del archivo separando sus valores por comas, devolviendo cada fila como
    ↪ una LISTA
#     de strings (una lista por línea).
# 4) for linea in data_reader recorre cada fila leída.
# 5) linea[0] y linea[1] acceden a la primera y segunda columna de cada fila
#     (Operacion y Descripcion), y se concatenan con ' ---- ' entre ambas.
# Resultado esperado:
# Operacion ---- Descripcion
# a + b ---- suma a y b
# a - b ---- resta a menos b
# a / b ---- a dividido
# a // b ---- a dividido entre b (quitando decimales)
# a % b ---- devuelve el resto de la division a/b (modulus)
# a * b ---- a multiplicado por b
# a ** b ---- a elevado a b
# Nota: en el PDF original aparecía "divisiÃ³n" en vez de "división"; eso
    ↪ ocurría porque
# el archivo original se guardó en una codificación y se leyó/imprimió
    ↪ asumiendo otra
# distinta (un problema clásico de encoding). Aquí, al crear y leer el archivo
    ↪ consistentemente
# en UTF-8, el texto se muestra correctamente.
```

```
Operacion ---- Descripcion
a + b ---- suma a y b
a - b ---- resta a menos b
a / b ---- a dividido
```



```

a // b ---- a dividido entre b (quitando decimales)
a % b ---- devuelve el resto de la división a/b (modulus)
a * b ---- a multiplicado por b
a ** b ---- a elevado a b

```

```

[11]: ruta = os.path.join("res", "tabla_operaciones.csv")
f = open(ruta, encoding="utf-8")
data_reader = csv.reader(f, delimiter=',')

ruta_o = os.path.join("res", "4_tabla_operaciones.csv")
with open(ruta_o, 'w', encoding="utf-8", newline='') as f2:
    csv_writer = csv.writer(f2, delimiter='|')
    for line in data_reader:
        print(line)
        csv_writer.writerow(line)

f.close()

# Explicación línea por línea:
# 1) Se abre el CSV original para lectura y se crea un csv.reader sobre él
#    ↪ (delimitador ',').
# 2) ruta_o apunta a un nuevo archivo de salida, '4_tabla_operaciones.csv'.
# 3) Se abre ese archivo de salida en modo escritura con 'newline=""' (buena
#    ↪ práctica al
#       escribir CSV, para evitar líneas en blanco extra en Windows).
# 4) csv.writer(f2, delimiter='|') crea un escritor CSV que usará '|' como
#    ↪ separador
#       (en vez de la coma original).
# 5) for line in data_reader recorre cada fila leída del CSV original (cada
#    ↪ 'line' es
#       una lista, ej: ['Operacion', 'Descripcion']).
# 6) print(line) muestra esa fila tal cual se leyó.
# 7) csv_writer.writerow(line) escribe esa misma fila en el nuevo archivo, pero
#    ↪ ahora
#       separada por '|' en lugar de ', '.
# 8) f.close() cierra el archivo de lectura (el de escritura se cierra
#    ↪ automáticamente
#       al salir del 'with').
# Resultado esperado (impreso en pantalla, una lista por fila):
# ['Operacion', 'Descripcion']
# ['a + b', 'suma a y b']
# ['a - b', 'resta a menos b']
# ['a / b', 'a dividido']
# ['a // b', 'a dividido entre b (quitando decimales)']
# ['a % b', 'devuelve el resto de la division a/b (modulus)']
# ['a * b', 'a multiplicado por b']
# ['a ** b', 'a elevado a b']

```

```
# Además, queda creado el archivo 'res/4_tabla_operaciones.csv' con el mismo
↪ contenido
# pero separado por '/' en vez de ','.

```

```
['Operacion', 'Descripcion']
['a + b', 'suma a y b']
['a - b', 'resta a menos b']
['a / b', 'a dividido']
['a // b', 'a dividido entre b (quitando decimales)']
['a % b', 'devuelve el resto de la división a/b (modulus)']
['a * b', 'a multiplicado por b']
['a ** b', 'a elevado a b']

```

2.5 5. Docstrings

- Python permite adjuntar documentación a los objetos e inspeccionarla a través de la línea de comandos o durante la ejecución del programa.
- Los docstrings se almacenan en el atributo `__doc__` de cada objeto.
- El valor de dicho atributo se puede mostrar por medio de la función `help`.

```
[12]: help(open)

# Explicación línea por línea:
# 1) help(open) muestra la documentación integrada de la función built-in
↪ 'open': su firma
# completa (parámetros como file, mode, buffering, encoding, etc.) y una
↪ descripción
# de su comportamiento.
# Resultado esperado: un bloque de texto de ayuda generado automáticamente por
↪ Python,
# describiendo la firma y el propósito de la función open(). El contenido
↪ exacto depende
# de la versión de Python instalada.

```

Help on function open in module io:

```
open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None,
closefd=True, opener=None)

```

Open file and return a stream. Raise OSError upon failure.

file is either a text or byte string giving the name (and the path if the file isn't in the current working directory) of the file to be opened or an integer file descriptor of the file to be wrapped. (If a file descriptor is given, it is closed when the returned I/O object is closed, unless closefd is set to False.)

mode is an optional string that specifies the mode in which the file is opened. It defaults to 'r' which means open for reading in text

mode. Other common values are 'w' for writing (truncating the file if it already exists), 'x' for creating and writing to a new file, and 'a' for appending (which on some Unix systems, means that all writes append to the end of the file regardless of the current seek position). In text mode, if encoding is not specified the encoding used is platform dependent: `locale.getencoding()` is called to get the current locale encoding.

(For reading and writing raw bytes use binary mode and leave encoding unspecified.) The available modes are:

=====	
Character Meaning	

'r'	open for reading (default)
'w'	open for writing, truncating the file first
'x'	create a new file and open it for writing
'a'	open for writing, appending to the end of the file if it exists
'b'	binary mode
't'	text mode (default)
'+'	open a disk file for updating (reading and writing)
=====	

The default mode is 'rt' (open for reading text). For binary random access, the mode 'w+b' opens and truncates the file to 0 bytes, while 'r+b' opens the file without truncation. The 'x' mode implies 'w' and raises an `FileExistsError` if the file already exists.

Python distinguishes between files opened in binary and text modes, even when the underlying operating system doesn't. Files opened in binary mode (appending 'b' to the mode argument) return contents as bytes objects without any decoding. In text mode (the default, or when 't' is appended to the mode argument), the contents of the file are returned as strings, the bytes having been first decoded using a platform-dependent encoding or using the specified encoding if given.

buffering is an optional integer used to set the buffering policy. Pass 0 to switch buffering off (only allowed in binary mode), 1 to select line buffering (only usable in text mode), and an integer > 1 to indicate the size of a fixed-size chunk buffer. When no buffering argument is given, the default buffering policy works as follows:

- * Binary files are buffered in fixed-size chunks; the size of the buffer is chosen using a heuristic trying to determine the underlying device's "block size" and falling back on `io.DEFAULT_BUFFER_SIZE`.
On many systems, the buffer will typically be 4096 or 8192 bytes long.
- * "Interactive" text files (files for which `isatty()` returns True) use line buffering. Other text files use the policy described above

for binary files.

encoding is the name of the encoding used to decode or encode the file. This should only be used in text mode. The default encoding is platform dependent, but any encoding supported by Python can be passed. See the codecs module for the list of supported encodings.

errors is an optional string that specifies how encoding errors are to be handled---this argument should not be used in binary mode. Pass 'strict' to raise a ValueError exception if there is an encoding error (the default of None has the same effect), or pass 'ignore' to ignore errors. (Note that ignoring encoding errors can lead to data loss.) See the documentation for codecs.register or run 'help(codecs.Codec)' for a list of the permitted encoding error strings.

newline controls how universal newlines works (it only applies to text mode). It can be None, '', '\n', '\r', and '\r\n'. It works as follows:

- * On input, if newline is None, universal newlines mode is enabled. Lines in the input can end in '\n', '\r', or '\r\n', and these are translated into '\n' before being returned to the caller. If it is '', universal newline mode is enabled, but line endings are returned to the caller untranslated. If it has any of the other legal values, input lines are only terminated by the given string, and the line ending is returned to the caller untranslated.
- * On output, if newline is None, any '\n' characters written are translated to the system default line separator, os.linesep. If newline is '' or '\n', no translation takes place. If newline is any of the other legal values, any '\n' characters written are translated to the given string.

If closefd is False, the underlying file descriptor will be kept open when the file is closed. This does not work when a file name is given and must be True in that case.

A custom opener can be used by passing a callable as *opener*. The underlying file descriptor for the file object is then obtained by calling *opener* with (*file*, *flags*). *opener* must return an open file descriptor (passing os.open as *opener* results in functionality similar to passing None).

open() returns a file object whose type depends on the mode, and through which the standard file operations such as reading and writing are performed. When open() is used to open a file in a text mode ('w', 'r', 'wt', 'rt', etc.), it returns a TextIOWrapper. When used to open a file in a binary mode, the returned class varies: in read binary

mode, it returns a `BufferedReader`; in write binary and append binary modes, it returns a `BufferedWriter`, and in read/write mode, it returns a `BufferedRandom`.

It is also possible to use a string or bytearray as a file for both reading and writing. For strings `StringIO` can be used like a file opened in a text mode, and for bytes a `BytesIO` can be used like a file opened in a binary mode.

- No es necesario editar el atributo `__doc__` directamente. Para asociar un docstring a un objeto basta con escribir el texto (entre triples comillas) al principio de módulos, funciones o clases, antes del código ejecutable.

```
[13]: def funcion_de_prueba():
        """Esta es la documentación de la función de prueba"""
        pass

help(funcion_de_prueba)

# Explicación línea por línea:
# 1) El string entre triples comillas justo después de 'def funcion_de_prueba():
    ↳ ' es su
#     DOCSTRING: Python lo detecta automáticamente y lo guarda en el atributo
    ↳ __doc__ de
#     la función.
# 2) 'pass' es el cuerpo de la función (no hace nada).
# 3) help(funcion_de_prueba) muestra esa documentación junto con la firma de la
    ↳ función.
# Resultado esperado (aproximado):
# Help on function funcion_de_prueba in module __main__:
#
# funcion_de_prueba()
#     Esta es la documentación de la función de prueba
```

Help on function funcion_de_prueba in module __main__:

```
funcion_de_prueba()
    Esta es la documentación de la función de prueba
```

- Accediendo directamente al atributo `__doc__` se obtiene solo el docstring (sin formato extra).

```
[14]: print(funcion_de_prueba.__doc__)

# Explicación línea por línea:
# 1) funcion_de_prueba.__doc__ accede directamente al atributo donde Python
    ↳ guardó el
#     docstring de la función definida en la celda anterior.
```

```
# 2) A diferencia de help(), esto NO agrega información adicional (firma,
↳ formato, etc.):
# solo devuelve el texto tal cual se escribió en las triples comillas.
# Resultado esperado: "Esta es la documentación de la función de prueba"
```

Esta es la documentación de la función de prueba

Para entidades más grandes, help muestra el docstring dividido en secciones.

```
[15]: import sys
help(sys)

# Explicación línea por línea:
# 1) import sys carga el módulo estándar 'sys'.
# 2) help(sys) muestra la documentación completa del módulo: su docstring
↳ general, y un
# listado organizado en secciones (funciones, clases, atributos, etc.)
↳ definidos en él.
# Resultado esperado: un bloque extenso de documentación con varias secciones
↳ (NAME,
# MODULE REFERENCE, DESCRIPTION, FUNCTIONS, etc.); el contenido exacto varía
↳ según la
# versión de Python.
```

Help on built-in module sys:

NAME

sys

MODULE REFERENCE

<https://docs.python.org/3.11/library/sys.html>

The following documentation is automatically generated from the Python source files. It may be incomplete, incorrect or include features that are considered implementation detail and may vary between Python implementations. When in doubt, consult the module reference at the location listed above.

DESCRIPTION

This module provides access to some objects used or maintained by the interpreter and to functions that interact strongly with the interpreter.

Dynamic objects:

argv -- command line arguments; argv[0] is the script pathname if known
path -- module search path; path[0] is the script directory, else ''
modules -- dictionary of loaded modules

displayhook -- called to show results in an interactive session
excepthook -- called to handle any uncaught exception other than SystemExit
 To customize printing in an interactive session or to install a custom
 top-level exception handler, assign other functions to replace these.

stdin -- standard input file object; used by input()
stdout -- standard output file object; used by print()
stderr -- standard error object; used for error messages
 By assigning other file objects (or objects that behave like files)
 to these, it is possible to redirect all of the interpreter's I/O.

last_type -- type of last uncaught exception
last_value -- value of last uncaught exception
last_traceback -- traceback of last uncaught exception
 These three are only available in an interactive session after a
 traceback has been printed.

Static objects:

builtin_module_names -- tuple of module names built into this interpreter
copyright -- copyright notice pertaining to this interpreter
exec_prefix -- prefix used to find the machine-specific Python library
executable -- absolute path of the executable binary of the Python
interpreter
float_info -- a named tuple with information about the float implementation.
float_repr_style -- string indicating the style of repr() output for floats
hash_info -- a named tuple with information about the hash algorithm.
hexversion -- version information encoded as a single integer
implementation -- Python implementation information.
int_info -- a named tuple with information about the int implementation.
maxsize -- the largest supported length of containers.
maxunicode -- the value of the largest Unicode code point
platform -- platform identifier
prefix -- prefix used to find the Python library
thread_info -- a named tuple with information about the thread
implementation.
version -- the version of this interpreter as a string
version_info -- version information as a named tuple
dllhandle -- [Windows only] integer handle of the Python DLL
winver -- [Windows only] version number of the Python DLL
_enablelegacywindowsfsencoding -- [Windows only]
__stdin__ -- the original stdin; don't touch!
__stdout__ -- the original stdout; don't touch!
__stderr__ -- the original stderr; don't touch!
__displayhook__ -- the original displayhook; don't touch!
__excepthook__ -- the original excepthook; don't touch!

Functions:

```

displayhook() -- print an object to the screen, and save it in builtins._
excepthook() -- print an exception and its traceback to sys.stderr
exception() -- return the current thread's active exception
exc_info() -- return information about the current thread's active exception
exit() -- exit the interpreter by raising SystemExit
getdlopenflags() -- returns flags to be used for dlopen() calls
getprofile() -- get the global profiling function
getrefcount() -- return the reference count for an object (plus one :-)
getrecursionlimit() -- return the max recursion depth for the interpreter
getsizeof() -- return the size of an object in bytes
gettrace() -- get the global debug tracing function
setdlopenflags() -- set the flags to be used for dlopen() calls
setprofile() -- set the global profiling function
setrecursionlimit() -- set the max recursion depth for the interpreter
settrace() -- set the global debug tracing function

```

FUNCTIONS

```

__breakpointhook__ = breakpointhook(...)
    breakpointhook(*args, **kws)

```

This hook function is called by built-in breakpoint().

```

__displayhook__ = displayhook(object, /)
    Print an object to sys.stdout and also save it in builtins._

```

```

__excepthook__ = excepthook(exctype, value, traceback, /)
    Handle an exception by displaying it with a traceback on sys.stderr.

```

```

__unraisablehook__ = unraisablehook(unraisable, /)
    Handle an unraisable exception.

```

The unraisable argument has the following attributes:

- * exc_type: Exception type.
- * exc_value: Exception value, can be None.
- * exc_traceback: Exception traceback, can be None.
- * err_msg: Error message, can be None.
- * object: Object causing the exception, can be None.

```

addaudithook(hook)
    Adds a new audit hook callback.

```

```

audit(...)
    audit(event, *args)

```

Passes the event to any audit hooks that are attached.

`call_tracing(func, args, /)`
 Call `func(*args)`, while tracing is enabled.

The tracing state is saved, and restored afterwards. This is intended to be called from a debugger from a checkpoint, to recursively debug some other code.

`exc_info()`
 Return current exception information: (type, value, traceback).

Return information about the most recent exception caught by an except clause in the current stack frame or in an older stack frame.

`exception()`
 Return the current exception.

Return the most recent exception caught by an except clause in the current stack frame or in an older stack frame, or None if no such exception exists.

`exit(status=None, /)`
 Exit the interpreter by raising `SystemExit(status)`.

If the status is omitted or None, it defaults to zero (i.e., success). If the status is an integer, it will be used as the system exit status. If it is another kind of object, it will be printed and the system exit status will be one (i.e., failure).

`get_asyncgen_hooks()`
 Return the installed asynchronous generators hooks.

This returns a namedtuple of the form (firstiter, finalizer).

`get_coroutine_origin_tracking_depth()`
 Check status of origin tracking for coroutine objects in this thread.

`get_int_max_str_digits()`
 Return the maximum string digits limit for non-binary int<->str conversions.

`getallocatedblocks()`
 Return the number of memory blocks currently allocated.

`getdefaultencoding()`
 Return the current default encoding used by the Unicode implementation.

`getfilesystemencodeerrors()`
 Return the error mode used Unicode to OS filename conversion.

`getfilesystemencoding()`

Return the encoding used to convert Unicode filenames to OS filenames.

`getprofile()`

Return the profiling function set with `sys.setprofile`.

See the profiler chapter in the library manual.

`getrecursionlimit()`

Return the current value of the recursion limit.

The recursion limit is the maximum depth of the Python interpreter stack. This limit prevents infinite recursion from causing an overflow of the C stack and crashing Python.

`getrefcount(object, /)`

Return the reference count of object.

The count returned is generally one higher than you might expect, because it includes the (temporary) reference as an argument to `getrefcount()`.

`getsizeof(...)`

`getsizeof(object [, default]) -> int`

Return the size of object in bytes.

`getswitchinterval()`

Return the current thread switch interval; see `sys.setswitchinterval()`.

`gettrace()`

Return the global debug tracing function set with `sys.settrace`.

See the debugger chapter in the library manual.

`getwindowsversion()`

Return info about the running version of Windows as a named tuple.

The members are named: `major`, `minor`, `build`, `platform`, `service_pack`, `service_pack_major`, `service_pack_minor`, `suite_mask`, `product_type` and `platform_version`. For backward compatibility, only the first 5 items are available by indexing. All elements are numbers, except `service_pack` and `platform_type` which are strings, and `platform_version` which is a 3-tuple. Platform is always 2. Product_type may be 1 for a workstation, 2 for a domain controller, 3 for a server. Platform_version is a 3-tuple containing a version number that is intended for identifying the OS rather than feature detection.

```
intern(string, /)
    ``Intern`` the given string.
```

This enters the string in the (global) table of interned strings whose purpose is to speed up dictionary lookups. Return the string itself or the previously interned string object with the same value.

```
is_finalizing()
    Return True if Python is exiting.
```

```
set_asyncgen_hooks(...)
    set_asyncgen_hooks([firstiter] [, finalizer])
```

Set a finalizer for async generators objects.

```
set_coroutine_origin_tracking_depth(depth)
    Enable or disable origin tracking for coroutine objects in this thread.
```

Coroutine objects will track 'depth' frames of traceback information about where they came from, available in their `cr_origin` attribute.

Set a depth of 0 to disable.

```
set_int_max_str_digits(maxdigits)
    Set the maximum string digits limit for non-binary int<->str
conversions.
```

```
setprofile(...)
    setprofile(function)
```

Set the profiling function. It will be called on each function call and return. See the profiler chapter in the library manual.

```
setrecursionlimit(limit, /)
    Set the maximum depth of the Python interpreter stack to n.
```

This limit prevents infinite recursion from causing an overflow of the C stack and crashing Python. The highest possible limit is platform-dependent.

```
setswitchinterval(interval, /)
    Set the ideal thread switching delay inside the Python interpreter.
```

The actual frequency of switching threads can be lower if the interpreter executes long sequences of uninterruptible code (this is implementation-specific and workload-dependent).

The parameter must represent the desired switching delay in seconds
A typical value is 0.005 (5 milliseconds).

```
settrace(...)
    settrace(function)
```

Set the global debug tracing function. It will be called on each
function call. See the debugger chapter in the library manual.

```
unraisablehook(unraisable, /)
    Handle an unraisable exception.
```

The unraisable argument has the following attributes:

- * `exc_type`: Exception type.
- * `exc_value`: Exception value, can be None.
- * `exc_traceback`: Exception traceback, can be None.
- * `err_msg`: Error message, can be None.
- * `object`: Object causing the exception, can be None.

DATA

```
__stderr__ = <_io.TextIOWrapper name='<stderr>' mode='w' encoding='cp1...
__stdin__ = <_io.TextIOWrapper name='<stdin>' mode='r' encoding='cp125...
__stdout__ = <_io.TextIOWrapper name='<stdout>' mode='w' encoding='cp1...
api_version = 1013
argv = [r'C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packag...
base_exec_prefix = r'C:\Users\USER\anaconda3\envs\MachineLearning'
base_prefix = r'C:\Users\USER\anaconda3\envs\MachineLearning'
builtin_module_names = ('_abc', '_ast', '_bisect', '_blake2', '_codecs...
byteorder = 'little'
copyright = 'Copyright (c) 2001-2023 Python Software Foundati...ematis...
displayhook = <ipykernel.displayhook.ZMQShellDisplayHook object>
dllhandle = 140732725198848
dont_write_bytecode = False
exec_prefix = r'C:\Users\USER\anaconda3\envs\MachineLearning'
executable = r'C:\Users\USER\anaconda3\envs\MachineLearning\python.exe...
flags = sys.flags(debug=0, inspect=0, interactive=0, opt...coding=0, s...
float_info = sys.float_info(max=1.7976931348623157e+308, max_...epsilo...
float_repr_style = 'short'
hash_info = sys.hash_info(width=64, modulus=2305843009213693...iphash1...
hexversion = 51056624
implementation = namespace(name='cpython', cache_tag='cpython-311...as...
int_info = sys.int_info(bits_per_digit=30, sizeof_digit=4, ..._str_dig...
maxsize = 9223372036854775807
maxunicode = 1114111
meta_path = [<_distutils_hack.DistutilsMetaFinder object>, <class '_fr...
modules = {'IPython': <module 'IPython' from 'C:\\Users\\USER\\anacond...
orig_argv = [r'C:/Users/USER/anaconda3/envs/MachineLearning/python.exe...
```

```

path = [r'C:\Users\USER\anaconda3\envs\MachineLearning\python311.zip',...
path_hooks = [<class 'zipimport.zipimporter'>, <function FileFinder.pa...
path_importer_cache = {r'C:\Users\USER\Codespaces\Machine Learning': F...
platform = 'win32'
platlibdir = 'DLLs'
prefix = r'C:\Users\USER\anaconda3\envs\MachineLearning'
ps1 = 'In : '
ps2 = '...: '
ps3 = 'Out: '
pycache_prefix = None
stderr = <ipykernel.iostream.OutStream object>
stdin = <_io.TextIOWrapper name='<stdin>' mode='r' encoding='cp1252'>
stdlib_module_names = frozenset({'__future__', '_abc', '_aix_support',...
stdout = <ipykernel.iostream.OutStream object>
thread_info = sys.thread_info(name='nt', lock=None, version=None)
version = '3.11.15 | packaged by Anaconda, Inc. | (main, Jun 11 2026, ...
version_info = sys.version_info(major=3, minor=11, micro=15, releasele...
warnoptions = []
winver = '3.11'

```

FILE

(built-in)

Los docstrings que ocupan más de una línea suelen tener estas partes: - Resumen en la primera línea. - Línea en blanco. - Descripción detallada. - Otra línea en blanco antes del código.

```

[16]: def funcion_de_prueba2():
        """Esta es la línea para el resumen.

        Este es el párrafo donde se puede escribir una descripción más
detallada de la función. Observa que el resumen y la descripción
detallada van separados por una línea en blanco. También hay
otra línea en blanco antes de empezar el código.
        """

        pass

help(funcion_de_prueba2)

# Explicación línea por línea:
# 1) El docstring de esta función sigue la convención de resumen + línea en
↳ blanco +
# descripción detallada, que es una práctica recomendada para documentar
↳ funciones
# de forma clara y consistente.
# 2) 'pass' es el cuerpo (vacío) de la función.

```

```
# 3) help(funcion_de_prueba2) muestra ese docstring completo, con su formato de
↪ párrafos.
# Resultado esperado (aproximado):
# Help on function funcion_de_prueba2 in module __main__:
#
# funcion_de_prueba2()
#     Esta es la línea para el resumen.
#
#     Este es el párrafo donde se puede escribir una descripción más
#     detallada de la función. Observa que el resumen y la descripción
#     detallada van separados por una línea en blanco. También hay
#     otra línea en blanco antes de empezar el código.
```

Help on function funcion_de_prueba2 in module __main__:

```
funcion_de_prueba2()
    Esta es la línea para el resumen.

    Este es el párrafo donde se puede escribir una descripción más
    detallada de la función. Observa que el resumen y la descripción
    detallada van separados por una línea en blanco. También hay
    otra línea en blanco antes de empezar el código.
```

Comentarios adicionales: - Comentarios con `#` suelen asociarse a expresiones sencillas, instrucciones individuales o pequeños bloques de código. - Los docstrings son más apropiados para construcciones de más alto nivel: funciones, clases y módulos.

2.5.1 Sphinx

- Herramienta de documentación, especialmente útil para sistemas grandes.
- Da soporte a una gran variedad de formatos de salida: HTML, LaTeX, ePub, etc.

2.5.2 Estilo

- Es importante que los docstrings sean consistentes.
- El equipo de desarrollo debe acordar un formato y seguirlo rigurosamente (por ejemplo, la *Google Python Style Guide*, con reglas para módulos, funciones/métodos y clases).

2.6 6. Ejercicios (enunciados)

1. Escribe una función que reciba una ruta de un fichero de texto y una cadena de caracteres a buscar, y determine si la cadena aparece en el fichero.
2. Escribe una función que reciba una lista, una ruta destino y un número n . La función debe crear un fichero en la ruta especificada. El contenido del fichero serán los primeros n elementos de la lista. La función debe controlar de manera apropiada los posibles valores de n que estén fuera de rango.
3. Escribe una función que reciba una ruta de un fichero de texto y devuelva un diccionario con la frecuencia de aparición de cada palabra. Ejemplo: un fichero que contenga la frase 'es mejor

que venga que que no venga' devolverá el diccionario {'es': 1, 'mejor': 1, 'que': 3, 'venga': 2, 'no': 1}. Para dividir un string en palabras se puede usar el método split.

2.7 7. Soluciones propuestas

```
[17]: def cadena_en_fichero(ruta, cadena):
    with open(ruta, encoding="utf-8") as fichero:
        contenido = fichero.read()
    return cadena in contenido

ruta_prueba = os.path.join("res", "multiple_lines.txt")
print(cadena_en_fichero(ruta_prueba, "Segunda"))
print(cadena_en_fichero(ruta_prueba, "Cuarta"))

# Explicación línea por línea (Ejercicio 1):
# 1) cadena_en_fichero(ruta, cadena) abre el fichero con un 'with' (se cierra
    ↪ automáticamente).
# 2) fichero.read() lee TODO el contenido en un único string.
# 3) 'cadena in contenido' usa el operador 'in' para comprobar si 'cadena'
    ↪ aparece como
#     substring dentro del contenido leído; retorna True o False.
# 4) Se prueba buscando "Segunda" (aparece en "Segunda línea del fichero") y
    ↪ "Cuarta"
#     (no aparece en ningún lado del archivo de ejemplo).
# Resultado esperado:
# True
# False
```

False

False

```
[18]: def crear_fichero_con_n_elementos(lista, ruta_destino, n):
    if n < 0 or n > len(lista):
        print(f"Error: n={n} está fuera de rango (0 a {len(lista)})")
        return
    elementos = lista[:n]
    with open(ruta_destino, 'w', encoding="utf-8") as fichero:
        for elemento in elementos:
            fichero.write(str(elemento) + '\n')
    print(f"Fichero creado con {n} elementos en {ruta_destino}")

lista_ejemplo = ['manzana', 'pera', 'uva', 'kiwi', 'mango']
ruta_salida = os.path.join("res", "primeros_elementos.txt")
crear_fichero_con_n_elementos(lista_ejemplo, ruta_salida, 3)
crear_fichero_con_n_elementos(lista_ejemplo, ruta_salida, 10)

# Explicación línea por línea (Ejercicio 2):
```

```

# 1) La función valida primero que 'n' esté en un rango válido: no puede ser
    ↪ negativo ni
#     mayor que la longitud de la lista; si está fuera de rango, imprime un
    ↪ error y
#     termina (return) sin crear el fichero.
# 2) 'lista[:n]' toma los primeros n elementos de la lista mediante slicing.
# 3) Se abre el fichero destino en modo escritura ('w'), lo que crea el archivo
    ↪ (o lo
#     sobrescribe si ya existía).
# 4) Se escribe cada elemento en una línea distinta, convertido a string por
    ↪ seguridad.
# 5) Se confirma con un mensaje cuántos elementos se escribieron.
# 6) La primera llamada usa n=3 (válido, dentro del rango 0-5) -> crea el
    ↪ fichero con
#     'manzana', 'pera', 'uva'.
# 7) La segunda llamada usa n=10, que excede la longitud de la lista (5) -> se
    ↪ detecta como
#     fuera de rango y NO se crea/sobrescribe el fichero.
# Resultado esperado:
# Fichero creado con 3 elementos en res/primeros_elementos.txt
# Error: n=10 está fuera de rango (0 a 5)

```

Fichero creado con 3 elementos en res\primeros_elementos.txt

Error: n=10 está fuera de rango (0 a 5)

```

[19]: def frecuencia_palabras(ruta):
        with open(ruta, encoding="utf-8") as fichero:
            contenido = fichero.read()
            palabras = contenido.split()
            frecuencias = {}
            for palabra in palabras:
                frecuencias[palabra] = frecuencias.get(palabra, 0) + 1
            return frecuencias

ruta_frase = os.path.join("res", "frase_ejemplo.txt")
with open(ruta_frase, 'w', encoding="utf-8") as f:
    f.write("es mejor que que no venga")

print(frecuencia_palabras(ruta_frase))

# Explicación línea por línea (Ejercicio 3):
# 1) Se abre el fichero y se lee todo su contenido en un string.
# 2) contenido.split() separa el string en una lista de palabras, usando
    ↪ espacios en
#     blanco como delimitador por defecto (también maneja saltos de línea y
    ↪ tabulaciones).
# 3) 'frecuencias' es un diccionario vacío que irá acumulando los conteos.

```



```
# 4) for palabra in palabras recorre cada palabra de la lista.
# 5) 'frecuencias.get(palabra, 0) + 1' obtiene el conteo actual de esa palabra
    ↪ (0 si es
#     la primera vez que aparece) y le suma 1; el resultado se guarda de vuelta
    ↪ en el
#     diccionario para esa clave.
# 6) Se crea primero un archivo de prueba con la frase del enunciado, y luego
    ↪ se invoca
#     la función sobre ese archivo.
# Resultado esperado: {'es': 1, 'mejor': 1, 'que': 3, 'venga': 2, 'no': 1}
```

```
{'es': 1, 'mejor': 1, 'que': 3, 'venga': 2, 'no': 1}
```